

Why Programming Still Matters in the Age of AI

Computer Science 2 — before we open the syllabus

Sumin Han

`suminhan@ksa.hs.kr`

Let's Ask It Properly

The question you are already thinking

“이제 AI가 코드를 짜준다. 나보다 빠르고, 솔직히 나보다 더 잘 짠다. 그런데 내가 왜 프로그래밍 수업에 앉아 있어야 하지?”

정당한 질문임. 그리고 이 질문에 “AI는 코딩을 잘 못한다”는 식으로 답할 생각은 없음.

못하지 않음. 아주 잘하고, 매달 더 좋아지고 있음.

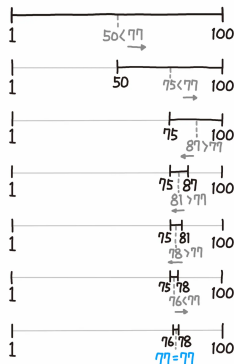
대신 다른 방식으로 답하려 함 — 그래서 어떤 일이 새로 생겼는지 보여주는 방식으로.

Yes — This Part Is Really True

이진 탐색을 짜달라고 하면 즉시 나옴. 그리고 맞음.

```
def binary_search(L, target):
    lo, hi = 0, len(L) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if L[mid] == target: return mid
        elif L[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
```

77을 찾아라



10년 전엔 한 주치 수업 분량. 지금은 4초. 그럼 무엇이 비싸졌는가?

The Bottleneck Moved

Before

어려운 부분: 코드를 **쓰는** 것.
하루 100줄을 썼고, 그 100줄을
전부 이해하고 있었음 — 쓰려면
이해할 수밖에 없으니까.

Now

어려운 부분: 코드를 **판단하는**
것.
1분이면 1000줄을 뺏음. **그중 몇
줄이나 이해하고 있나?**

AI가 좋아진다고 이 일이 줄지 않음.
오히려 늘어남.

This Is Already Showing Up — 2026

The first step is squeezed

신입과 인턴은 취업하기 더 어려워졌음.

빅테크의 평균 연차는 계속 올라가는 중임.

But demand is not shrinking

상위 테크 기업 채용은 1년 전보다 약 20% 증가.

AI 엔지니어링 공고: Apple, Google 등에서 50-100% 증가.

출처: The Pragmatic Engineer, "State of the Software Engineering Job Market" (2026, TrueUp 채용 데이터) — 미국 시장 기준.

둘 다 동시에 참임. 왜?

예전에 입문하는 방법은 "작고 명확하게 정의된 과제를 줄 테니 구현해와라"였음. **그건 지금 AI가 가장 잘하는 일임. 그 첫 계단이 사라졌음.** 남은 일은 한 단계 위에서 시작함 — 결정하고, 지시하고, 검증하는 일.

"But I Can Just Ask in Korean"

맞음. 한국어로 물어봐도 되고 영어로 물어봐도 됨. 그렇게 하면 됨.

But look at which direction that helps

들어갈 때는 사람의 언어를 쓸 수 있음.

돌아올 때는 **답이 항상 코드임**. 출력에는 한국어 버전이 없음. 오직 코드가 있을 뿐이고, 그걸 읽을 수 있느냐만 남음.

돌아온 것을 읽지 못한다면,
지시를 한 게 아니라 —
소원을 빈 것임.

“Write Me a Grade Calculator”

그래서 시켜봤음. 이렇게 나왔음 — 깔끔하고, 읽기 쉽고, 잘 돌아감:

```
def final_grade(in_class, assignments, final_exam,
               attitude, attend_rate):
    if attend_rate < 0.75:
        return "Fail"
    score = (0.2*in_class + 0.2*assignments
            + 0.5*final_exam + 0.1*attitude)
    return "Pass" if score >= 80 else "Fail"
```

맞아 보임. 돌아감. 에러도 없음. 그런데 이 수업 기준으로는 틀렸음.

중요한 건 이 부분임: 어디를 봐도 틀려 보이지 않는다는 것.
경고해주지 않음. 그냥 조용히 잘못된 성적을 계산할 뿐임.

Three Things It Could Not Have Known

- 어떤 학생이 **병결**로 기말고사를 못 봤음. 이 과목은 **중간고사가 없어서** 인정점수를 계산할 다른 시험 점수 자체가 없음 — 그래서 규정상 무조건 **재평가**를 봐야 함. 코드는 그런 예외를 들어본 적이 없음.
- 그 **재평가**도 병결/공결 예외를 **원칙적으로 인정 안 함** (학부회의를 거쳐야 예외 조정 가능). 재평가마저 결시하면 **그냥 0점**. 코드는 “재평가”라는 개념 자체를 모름.
- 출석 3/4 미만 — 이게 모든 걸 덮어쓰는가, 아니면 먼저 확인하는가? 코드는 찍었음. **그리고 우리와 다르게 찍었음.**

“그럼 규정을 프롬프트에 넣으면 되잖아요!” — 맞음. **그런데 그렇게 하려면 뭐가 빠졌는지를 이미 알아챌어야 함.** 코드는 알려주지 않았음. 구멍을 보려면 읽는 수밖에 없었음.

"I Tested It. It Works."

과제: 어떤 연도가 윤년이면 True 반환.

```
def is_leap_year(year):  
    if year % 4 == 0:  
        return True  
    else:  
        return False
```

(%는 나머지 연산 — $\text{year} \% 4 == 0$ 은 "4로 나누어떨어진다"는 뜻임.)

테스트해봄:

`is_leap_year(2024)`

⇒ True 정답!

통과했음. 이대로 제출?

Now Try This One

```
is_leap_year(1900)
```

기대값: False

1900년은 윤년이 아니었음.

Execution Result

True

왜: 진짜 규칙은 세 부분임 — **4**로 나누어떨어지고, 단 100으로 나누어떨어지면 제외하되, 그중 **400**으로 나누어떨어지면 다시 윤년. 코드는 첫 부분만 알고 있음. 1900은 100으로는 나뉘지만 400으로는 안 나뉘므로 윤년이 **아님**.

The part that should worry you

버그는 처음부터 거기 있었음. 내 테스트가 그쪽을 안 봤을 뿐임. **테스트를 통과했다는 건 맞다는 뜻이 아님.**

어떤 입력이 이걸 깨뜨리는지 고르는 것 — 그게 실력이었음. 그리고 그건 아무도 대신 해줄 수 없었음.

September 1999: Ten Months, Then Silence

NASA의 Mars Climate Orbiter는 10개월을 날아갔음. 도착하면 화성 상공 약 150 km에 안착할 예정이었음.

실제로는 57 km로 진입 — 대기권 한복판이었음. 그대로 부서졌고, 다시는 신호가 오지 않았음.

What went wrong

한 팀은 추력 값을 파운드 힘(lbf) 단위로 넘겼음. 받는 팀은 그걸 뉴턴(N)으로 읽었음.

1 lbf = 4.45 N이라, 모든 궤도 수정이 4.45배씩 어긋난 채로 — 10개월 동안. 손실: 1억 2500만 달러.

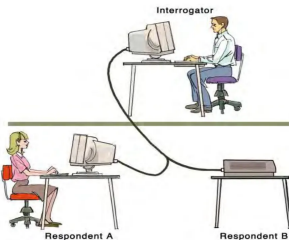
그 코드는 한 줄도 빠짐없이 완벽하게 돌아갔음. 그 숫자가 무슨 뜻인지 아무도 확인하지 않았을 뿐임.

1950: The Chair Turing Left Empty



Alan Turing, 1936.

The Father of the Computer.



The Imitation Game (1950).

튜링은 "기계가 생각할 수 있는가?"라고 물었고 — 답하기를 거부했음. 대신 시험으로 바꿈: **사람 심판(interrogator)**이 양쪽에 질문을 던지고 그 답이 말이 되는지 판단함.

컴퓨터의 맨 처음 설계에 이미 질문을 던지는 사람이 의자에 앉아 있었음. **그 의자는 끝내 자동화되지 않았음.**

This Is What the Syllabus Actually Asks of You

이 과목의 교과 핵심역량 네 가지 — 그리고 방금 각각을 어디서 만났는지:

- **Fundamental Programming & Problem-Solving**
돌아온 코드를 읽고, 그게 실제로 무슨 일을 하는지 이해하기.
- **Logical Thinking & Analytical Skills**
어떤 입력이 이걸 깨뜨릴지 — 깨지기 전에 알기.
- **Communication & Collaboration**
수업 시간에 문제를 같이, 소리 내어 푸는 이유.
- **Ethical Use of AI & Responsible Attitude**
도구는 쓰되, **답에 대한 책임은 내가 짐**. “AI가 짰는데요”는 변명이 되지 않음.

So — Why Are You Here?

- 기계보다 코드를 빨리 치려고 온 게 **아님**.
- 원하는 것을 정확히 말할 수 있고, 돌아온 것을 읽을 수 있고, 어떤 입력이 깨뜨릴지 아는 사람이 되기 위해서임.

So here is how this class runs

수업 시간에 문제는 AI 없이, 다 같이, 손으로 푸는 게 원칙임. AI를 쓰는 게 부정행위라서가 아니라 — 기계가 대신 해주는 능력은 내 것이 되지 않기 때문임.

평생 계산기를 쓸 것임.
그래도 산수는 손으로 배웠음.